



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

Binary Image Classification on Synthetic Scenes

Aayush Ranjan (CS25MTECH11002)

Ankit Kr Sinha (CS25MTECH11022)

Mayank Mishra (CS25MTECH11029)

CS5480: Deep Learning

2 May 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Dataset Observations	3
1.3	What Makes the Task Hard	3
1.4	What Makes it Tractable	4
1.5	Our Approach	4
2	Dataset Description	4
2.1	Overview	4
2.2	Dataset Statistics	4
2.3	Directory Structure and Loading	4
2.4	Train/Validation Split	5
3	Methodology	5
3.1	Problem Framing and Design Principles	5
3.2	System Architecture	5
3.3	Two-Round Self-Training with Pseudo-Labels	7
3.4	Key Design Decisions	7
4	Shared Model Architecture: ShapeNet	7
4.1	Design Philosophy	7
4.2	Building Blocks	8
4.2.1	Residual Block (ResBlock)	8
4.2.2	Squeeze-and-Excitation Block (SEBlock)	8
4.3	Full Architecture	8
5	Edge Feature Pipeline	9
5.1	Motivation	9
5.2	EdgeTransform Pipeline	9
5.3	Dataset Classes	10
5.4	Edge Model Training Configuration	10
6	RGB Model Pipeline	10
6.1	Motivation	10
6.2	Data Augmentation Strategy	11
6.3	Mixup Augmentation	11
6.4	Stochastic Weight Averaging (SWA)	11
6.5	RGB Model Training Configuration	12
7	Inference Pipeline	12
7.1	Pseudo-Label Selection	12
7.2	Test-Time Batch Normalisation Adaptation	12

7.3	Test-Time Augmentation (TTA)	13
7.4	Ensemble and Final Prediction	14
8	Ablation Study: Approaches Tried and Lessons Learned	14
8.1	A1 — Baseline: Custom ResNet-Style CNN (Score: 0.7534)	15
8.2	A2 — SE Blocks + Stronger Colour Augmentation (Score: ≈ 0.764)	15
8.3	A3 — Edge Stream + Dual-Stream Ensemble + BCE (Score: ≈ 0.769)	15
8.4	A4 — SWA + BN Test-Time Adaptation (Score: ≈ 0.787)	16
8.5	Final — Pseudo-Labelling + 4-way TTA (Score: 0.8092)	16
9	Results	17
9.1	Leaderboard Score	17
9.2	Validation Performance	17
9.3	Analysis of the Train–Test Gap	17
10	Team Contributions	18
11	Conclusion and Future Work	19
11.1	Summary	19
11.2	Analysis of the Remaining Gap	19
11.3	Future Work	19
	References	19

1 Introduction

1.1 Problem Statement

This report documents our solution to the *IITH Deep Learning 2026 Hackathon* hosted on Kaggle, offered as part of the CS5480 Deep Learning course at IIT Hyderabad. The task is binary image classification on a synthetic dataset of 3-D rendered scenes. Each scene contains objects of various shapes (cube, sphere, cylinder), materials (rubber/matte, metal/specular), sizes (small, large), and colours. Each image must be assigned a label of 0 (negative) or 1 (positive). The underlying classification rule is latent — it is not explicitly disclosed and must be inferred entirely from the training data.

A strict competition rule prohibits the use of any pre-trained model weights; all models must be trained from scratch on the provided dataset. Our final submission achieves a **public leaderboard accuracy of 0.8092** (exact score: 0.809226).

1.2 Dataset Observations

Several properties of this dataset are immediately apparent from inspection:

- **Perfectly balanced classes.** Both training classes contain exactly 9,000 images, so there is no class frequency prior to exploit. A model must learn the actual discriminating attribute rather than relying on label frequency.
- **Two material types with radically different appearances.** Rubber objects are matte and diffuse; metallic objects produce sharp specular highlights. This means the same object shape can look very different depending on its material. A model trained on raw RGB values may inadvertently learn to classify by material reflectance rather than by the true rule.
- **Controlled, renderer-generated variation.** Because scenes are synthetically rendered, the variation between images is fully governed by the object attribute sampler. If the sampler is not uniform over all attribute combinations, the training distribution will be biased — a model can achieve high training accuracy by exploiting whatever attribute is most correlated with the label in the training set, even if that attribute is not the true rule.
- **No canonical orientation.** Objects are placed freely in the scene; there is no fixed viewpoint or consistent up-direction. This makes rotational and flip invariances important.

1.3 What Makes the Task Hard

The central difficulty of this dataset is the **latent rule problem**: the label is generated by a deterministic function of the scene attributes, but that function is unknown. Near-perfect validation accuracy is achievable because the model can latch onto any proxy attribute that is perfectly correlated with the label in training. However, if the test set samples the attribute space more uniformly, or if the proxy attribute does not generalise, the leaderboard score will be much lower.

This explains the large gap we observe: both our RGB and Edge models reach ≈ 1.0 validation accuracy but only 0.8092 on the public leaderboard. The model likely learnt a spurious correlation — for example, a colour-label or material-label association present in training but weaker in the test set.

1.4 What Makes it Tractable

Despite the latent rule challenge, the dataset is tractable for several reasons. The scene rendering is clean and consistent, so there is no noise or domain randomness beyond the controlled attribute variation. The object geometry is simple and well-separated in images. Edge-based representations can therefore recover shape and size cues reliably without relying on appearance. The balanced training set also means a strong model should be able to separate the classes if the right features are used.

1.5 Our Approach

Our solution is a dual-stream convolutional pipeline. One stream operates on raw RGB pixel values; the other processes multi-scale edge-feature maps via a custom `EdgeTransform`. Both streams use the same custom backbone (**ShapeNet**) trained from scratch, combined via equal-weight logit-space averaging at inference. To further close the train-test gap, we employ two-round self-training with pseudo-labelled negatives, test-time batch normalisation adaptation, and four-way deterministic test-time augmentation (TTA).

2 Dataset Description

2.1 Overview

The dataset consists of synthetic PNG images of 3-D rendered scenes provided as part of the IITH Deep Learning 2026 Hackathon. Each scene contains objects varying along four attribute dimensions: *shape* (cube, sphere, cylinder), *material* (rubber/matte, metal/specular), *size* (small, large), and *colour*. Labels are binary: 0 (negative) and 1 (positive). The dataset is partitioned into a labelled training set, a public test set, and a private test set.

2.2 Dataset Statistics

Table 1: Dataset split statistics as provided in the competition description.

Split	Details	Images
Train — Class 0 (negative)	<code>train/0/</code>	9,000
Train — Class 1 (positive)	<code>train/1/</code>	9,000
Total training images	Perfectly balanced	18,000
Test (public + private)	<code>test/</code> (labels hidden, single folder)	5,010

2.3 Directory Structure and Loading

```
<data_dir>/
  train/
    0/      <- 9,000 class-0 (negative) images  [.png]
    1/      <- 9,000 class-1 (positive) images  [.png]
  test/     <- 5,010 unlabelled images           [.png]
```

The helper `_resolve_dir()` handles doubly-nested Kaggle extraction layouts. Training paths and labels are collected by `_load_train_paths()`. Test paths are gathered via a sorted `glob` of `*.png` files.

2.4 Train/Validation Split

A seeded (NumPy seed = 42) random 85%/15% shuffle is applied at the start of every training run:

```
1 np.random.seed(42)
2 np.random.shuffle(indices)
3 val_size = int(0.15 * n)
4 val_idx, train_idx = indices[:val_size], indices[val_size:]
```

Listing 1: Train/validation split (from code).

With 18,000 images this yields $\approx 15,300$ training and $\approx 2,700$ validation samples. The fixed seed ensures both RGB and Edge runs see identical validation splits.

3 Methodology

3.1 Problem Framing and Design Principles

The synthetic scene dataset presents two main challenges. First, the discriminating attribute is latent: a model can achieve near-perfect training accuracy by exploiting any correlated feature — even one that does not reflect the true classification rule. Second, metallic objects produce specular highlights that introduce sharp, material-dependent intensity gradients, which can mislead any model trained on raw RGB.

Our solution addresses both challenges through a **complementary dual-stream design**:

- The **RGB stream** retains full colour, texture, and lighting information, allowing the model to capture colour-based or appearance-based discriminating rules.
- The **Edge stream** discards appearance by converting each image to a multi-scale edge representation, focusing on geometric structure (shape outlines, object size, boundaries) in a material-invariant way.

Both streams use the same ShapeNet backbone, trained from scratch, with stream-specific augmentation. Their logits are combined via an equal-weight ensemble.

3.2 System Architecture

Figure 1 shows the complete end-to-end system architecture, from raw input data through to the final `submission.csv`.

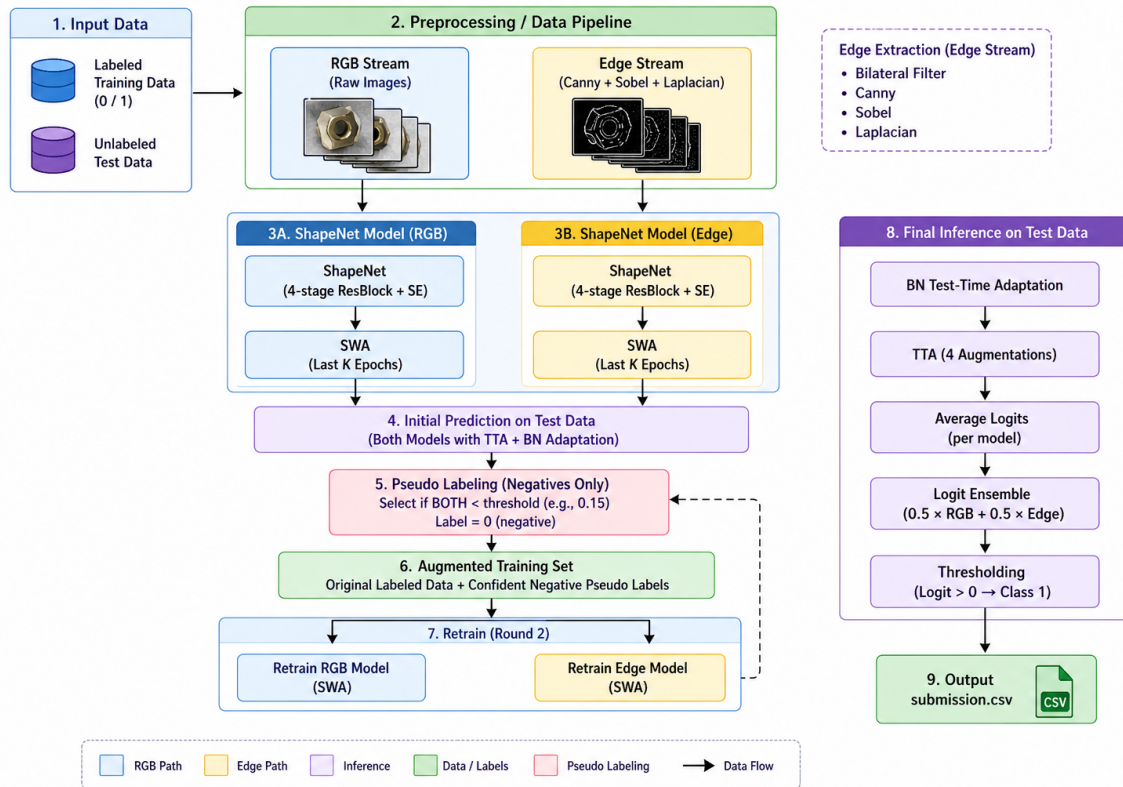


Figure 1: End-to-end dual-stream pipeline. The RGB path (blue) and Edge path (yellow) train ShapeNet independently. After Round 1, high-confidence test negatives are pseudo-labelled and added to the training pool for Round 2. Final predictions combine both streams via logit-space ensembling.

The pipeline proceeds in nine numbered stages as shown in the figure:

- 1. Input Data.** Labelled training images (18,000) and unlabelled test images (5,010).
- 2. Preprocessing / Data Pipeline.** Raw images feed the RGB stream unchanged; the Edge stream applies EdgeTransform (bilateral filter → Canny → Sobel → Laplacian).
- 3. Model Training (Round 1).** Two ShapeNet instances (3A: RGB, 3B: Edge) are trained independently, each with SWA over the last K epochs.
- 4. Initial Test Prediction.** Both Round 1 models predict on the test set with BN adaptation and 4-way TTA.
- 5. Pseudo-Labeling.** Test images where both models produce sigmoid probability < 0.15 are assigned pseudo-label 0 (negative).
- 6. Augmented Training Set.** Original labelled data combined with confident pseudo-negatives.
- 7. Round 2 Retraining.** Both models are retrained from scratch on the augmented set.
- 8. Final Inference.** BN adaptation → TTA → logit averaging per model → ensemble.
- 9. Output.** `submission.csv` with columns ID and Label.

3.3 Two-Round Self-Training with Pseudo-Labels

A single pass on the labelled set leaves the models blind to the test distribution. We adopt a two-round self-training loop:

1. **Round 1:** Train both models on the 18,000 labelled images. BN-adapt and TTA-score all 5,010 test images. Select confident negatives as pseudo-labels.
2. **Round 2:** Retrain both models from scratch on the expanded dataset. Apply BN adaptation, TTA, and ensemble to generate the final submission.

3.4 Key Design Decisions

Table 2: Summary of key design decisions and rationale.

Decision	Implementation	Rationale
No pre-trained weights	ShapeNet randomly initialised	Competition rule; ImageNet priors may not transfer to this synthetic domain
Edge stream	Bilateral filter $\rightarrow$ Canny, Sobel, Laplacian	Material-invariant geometry; suppresses specular artefacts
SWA	AveragedModel over last K epochs + update_bn pass	Flatter loss-surface minima; implicit ensemble effect
BN test-time adaptation	Reset & re-estimate BN stats on test set	Corrects distribution shift in batch statistics
Pseudo-labels (negatives only)	Dual threshold $\sigma < 0.15$ on both models	Expands negative pool conservatively; avoids false positives
4-way deterministic TTA	Identity, H-flip, V-flip, both	Reduces variance; scenes lack a canonical orientation
Equal-weight ensemble	$0.5 l_{\text{RGB}} + 0.5 l_{\text{Edge}}$	Complementary streams; no held-out set for tuning weights

4 Shared Model Architecture: ShapeNet

4.1 Design Philosophy

Both the RGB and Edge models are instantiated from the same custom architecture, **ShapeNet**: a compact four-stage convolutional encoder with residual connections and channel attention, followed by global average pooling and a two-layer classification head. No pre-trained weights are used.

Since discriminating features in this dataset are likely global (object shape, size, spatial arrangement) rather than fine-grained texture, a compact architecture with strong regularisation (Dropout, weight decay) is preferred over a large pre-trained network.

4.2 Building Blocks

4.2.1 Residual Block (ResBlock)

Each downsampling stage begins with a `ResBlock`: two 3×3 convolutions each followed by BatchNorm and ReLU, with an identity skip connection:

$$\mathbf{y} = \text{ReLU}(\mathbf{x} + \text{BN}(\text{Conv}_{3 \times 3}(\text{ReLU}(\text{BN}(\text{Conv}_{3 \times 3}(\mathbf{x})))))) \quad (1)$$

Since channel count is constant within each `ResBlock`, no projection shortcut is needed. The skip connection supports stable gradient flow across depth.

4.2.2 Squeeze-and-Excitation Block (SEBlock)

After each `ResBlock`, a `SEBlock` performs channel-wise recalibration via a two-layer FC bottleneck (reduction ratio $r = 4$) with sigmoid output:

$$\mathbf{s} = \sigma(W_2 \text{ReLU}(W_1 \text{GAP}(\mathbf{x}))), \quad \hat{\mathbf{x}} = \mathbf{s} \odot \mathbf{x} \quad (2)$$

This allows the network to up-weight discriminative channels (e.g. encoding shape outlines) and suppress uninformative ones (e.g. background gradients).

4.3 Full Architecture

Table 3: ShapeNet layer-by-layer summary. Inputs are resized to 192×192 .

Stage	Components	Out ch.	Spatial
Stem	Conv 3×3 , BN, ReLU	32	192^2
Down-1	ResBlock(32) $\rightarrow$ SEBlock(32) $\rightarrow$ Conv 3×3 s=2, BN, ReLU	64	96^2
Down-2	ResBlock(64) $\rightarrow$ SEBlock(64) $\rightarrow$ Conv 3×3 s=2, BN, ReLU	128	48^2
Down-3	ResBlock(128) $\rightarrow$ SEBlock(128) $\rightarrow$ Conv 3×3 s=2, BN, ReLU	256	24^2
Down-4	ResBlock(256) $\rightarrow$ SEBlock(256) $\rightarrow$ Conv 3×3 s=2, BN, ReLU	512	12^2
Pool	AdaptiveAvgPool2d(1)	512	1^2
Head	Flatten $\rightarrow$ Linear(512,256) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear(256,1)	1	—

The output is a single scalar logit. `BCEWithLogitsLoss` is used as the loss function. `AdaptiveAvgPool2d(1)` makes the classifier resolution-agnostic.

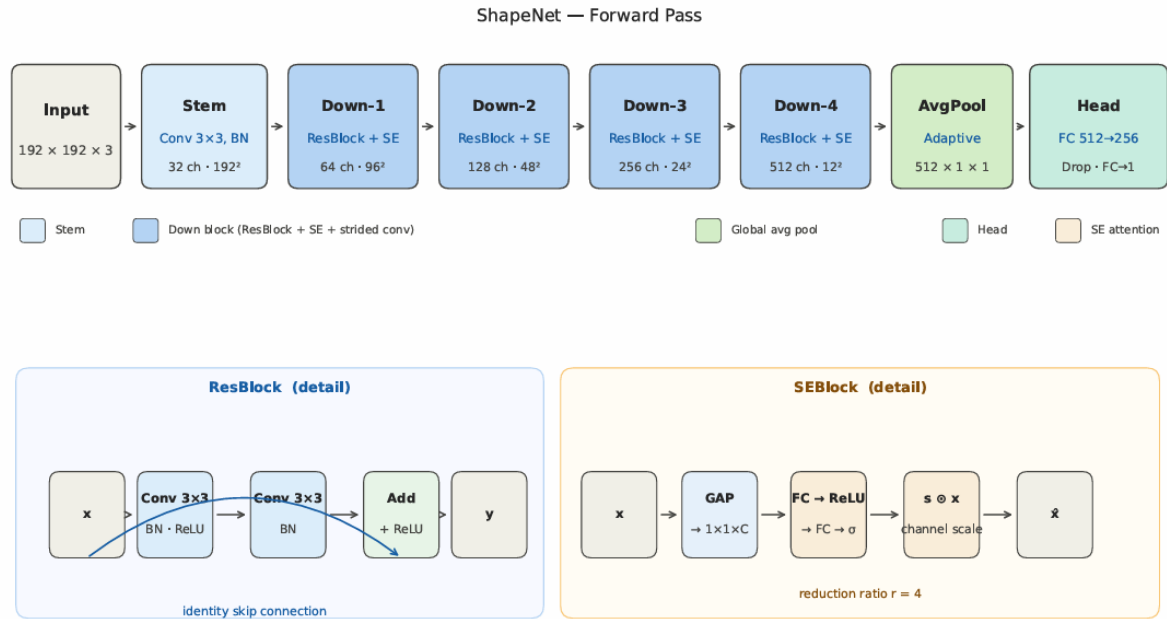


Figure 2: ShapeNet architecture block diagram. *Top row*: data flow from input through the stem, four downsampling blocks (each containing ResBlock + SEBlock + strided conv), global average pool, and the classification head. *Bottom row*: expanded internal structure of the ResBlock (identity skip connection over two conv layers) and the SEBlock (GAP → FC bottleneck → sigmoid scale → channel-wise multiplication $\odot$).

5 Edge Feature Pipeline

5.1 Motivation

The dataset includes two material types: rubber (diffuse, matte) and metal (specular, mirror-like). Metallic objects produce strong specular highlight gradients that are material-dependent rather than geometry-dependent. Training on raw RGB may cause the model to associate these artefacts with class labels rather than learning true shape or size differences.

The **EdgeTransform** converts each image into a three-channel edge map that captures geometric structure while suppressing material-related appearance variation.

5.2 EdgeTransform Pipeline

`EdgeTransform.call(img)` processes each PIL image as follows:

1. **Grayscale.** `img.convert('L')` → single-channel luminance array.
2. **Bilateral filter.** `cv2.bilateralFilter(arr, 9, 150, 150)`: aggressively smooths specular highlights while preserving sharp transitions at object boundaries.
3. **Canny edges.** `cv2.Canny(arr, 30, 100)`: binary boundary map normalised to $[0, 1]$.
4. **Sobel magnitude.** $G = \sqrt{G_x^2 + G_y^2}$, normalised to $[0, 1]$.
5. **Laplacian.** `cv2.Laplacian(ksize=3)`: second-order response, absolute value normalised to $[0, 1]$.

6. **Channel stacking.** Canny, Sobel, and Laplacian maps stacked as a $3 \times H \times W$ tensor, compatible with ShapeNet’s three-channel input.

The three channels are complementary: Canny gives clean binary boundaries, Sobel encodes continuous gradient strength, and Laplacian captures corners and fine structure.

5.3 Dataset Classes

`TrainDataset` and `TestDataset` accept an optional `edge_transform`. When set, it is applied to the raw `PIL` image *before* the torchvision pipeline, allowing both streams to share `train_one_model()` unchanged.

5.4 Edge Model Training Configuration

Table 4: Edge model training configuration.

Parameter	Value
Image size	192×192
Augmentations	<code>RandomHorizontalFlip</code> , <code>RandomVerticalFlip</code> , <code>RandomRotation($\pm 15^\circ$)</code>
Colour augmentations	None (irrelevant for edge maps)
Mixup	Disabled (<code>use_mixup=False</code>)
Epochs (Round 1 & 2)	40
SWA window	Last 10 epochs (starts at epoch 31)
Early stopping	Disabled (<code>patience_limit=40</code>)
Optimiser	<code>AdamW</code> ($\text{lr} = 10^{-3}$, $\text{wd} = 5 \times 10^{-3}$)
LR schedule	<code>CosineAnnealingLR</code> ($T_{\text{max}} = 40$)
Positive class weight	0.6
Batch size	96

Mixup is disabled for the Edge model because blending two edge maps produces a geometrically incoherent superposition. Early stopping is disabled so the full SWA window always completes.

6 RGB Model Pipeline

6.1 Motivation

The RGB model trains on raw pixel values, retaining full colour, texture, and lighting information. If the latent rule is colour-dependent, the RGB stream is better positioned to capture it than the edge stream. The two streams are deliberately complementary: the Edge model focuses on material-invariant geometry; the RGB model captures appearance cues the Edge model discards.

6.2 Data Augmentation Strategy

Table 5: RGB train-time augmentation pipeline (applied in order).

Transform	Parameters and purpose
Resize	192×192
RandomHorizontalFlip	$p = 0.5$. Left-right invariance.
RandomVerticalFlip	$p = 0.5$. Up-down invariance.
RandomRotation	$\pm 20^\circ$. Rotational invariance.
ColorJitter	Brightness, contrast, saturation ± 0.5 ; hue ± 0.2 . Colour invariance.
RandomGrayscale	$p = 0.3$. Encourages shape-based over colour-based features.
GaussianBlur	Kernel=7, $\sigma \in [0.5, 3.0]$, $p = 0.3$. Simulates focus variation.
RandomAutocontrast	$p = 0.2$. Lighting robustness.
ToTensor	Converts to $[0, 1]$ float tensor.
Normalize	Mean = $[0.5, 0.5, 0.5]$, Std = $[0.5, 0.5, 0.5]$. Maps to $[-1, 1]$.
RandomErasing	$p = 0.15$, scale $\in [0.02, 0.15]$. Simulates occlusion.

The evaluation/inference transform applies only Resize, ToTensor, and Normalize.

6.3 Mixup Augmentation

Mixup is applied within each training batch (`use_mixup=True`, RGB only). A mixing coefficient $\lambda \sim \text{Beta}(0.2, 0.2)$ forms convex combinations of image pairs and their labels:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j \quad (3)$$

Soft targets are handled natively by `BCEWithLogitsLoss`. Mixup smooths the decision boundary and reduces overconfident predictions.

6.4 Stochastic Weight Averaging (SWA)

Both models use SWA via PyTorch’s `AveragedModel`. At every epoch $e \geq T - K + 1$, the running average is updated:

$$\theta_{\text{SWA}}^{(e)} = \frac{(e - T_{\text{start}}) \theta_{\text{SWA}}^{(e-1)} + \theta^{(e)}}{e - T_{\text{start}} + 1} \quad (4)$$

For the RGB model, $K = 15$ and $T = 60$, so SWA begins at epoch 46. After accumulation, BN statistics are recomputed via `update_bn`. The SWA weights are kept if they match or exceed the best single-epoch checkpoint; otherwise the best checkpoint is restored.

6.5 RGB Model Training Configuration

Table 6: RGB model training configuration.

Parameter	Value
Image size	192×192
Batch size	96
Optimiser	AdamW ($\text{lr} = 10^{-3}$, $\text{wd} = 5 \times 10^{-3}$)
LR schedule	CosineAnnealingLR ($T_{\text{max}} = 60$)
Epochs (Round 1 & 2)	60
SWA window	Last 15 epochs (starts at epoch 46)
Positive class weight	0.6
Mixup	Enabled ($\alpha = 0.2$)
Early stopping	Disabled (<code>patience_limit=60</code>)

7 Inference Pipeline

7.1 Pseudo-Label Selection

After Round 1 training, each model produces logits on all 5,010 test images after BN adaptation and TTA. A test image is pseudo-labelled as negative (label = 0) only when *both* models agree with high confidence:

$$\text{pseudo-negative} \Leftrightarrow \sigma\left(l_{\text{RGB}}^{(1)}\right) < 0.15 \wedge \sigma\left(l_{\text{Edge}}^{(1)}\right) < 0.15 \quad (5)$$

Only negative pseudo-labels are produced. A false positive pseudo-label would inflate the positive prediction rate in Round 2, while a spurious negative only wastes a training sample. Dual-model agreement further reduces noise. GPU memory is freed between rounds via `del` and `torch.cuda.empty_cache()`.

7.2 Test-Time Batch Normalisation Adaptation

BatchNorm running statistics estimated during training may not reflect the test distribution. `adapt_bn()` corrects this by:

1. Setting all BatchNorm2d modules to `train()` mode and resetting running statistics via `reset_running_stats()`.
2. Running a full forward pass over the test set (`shuffle=True, no_grad()`) to re-accumulate statistics from test batches.
3. Returning all BN modules to `eval()` mode.

BN adaptation is applied before every inference pass: once before Round 1 pseudo-label scoring and once before final Round 2 inference.

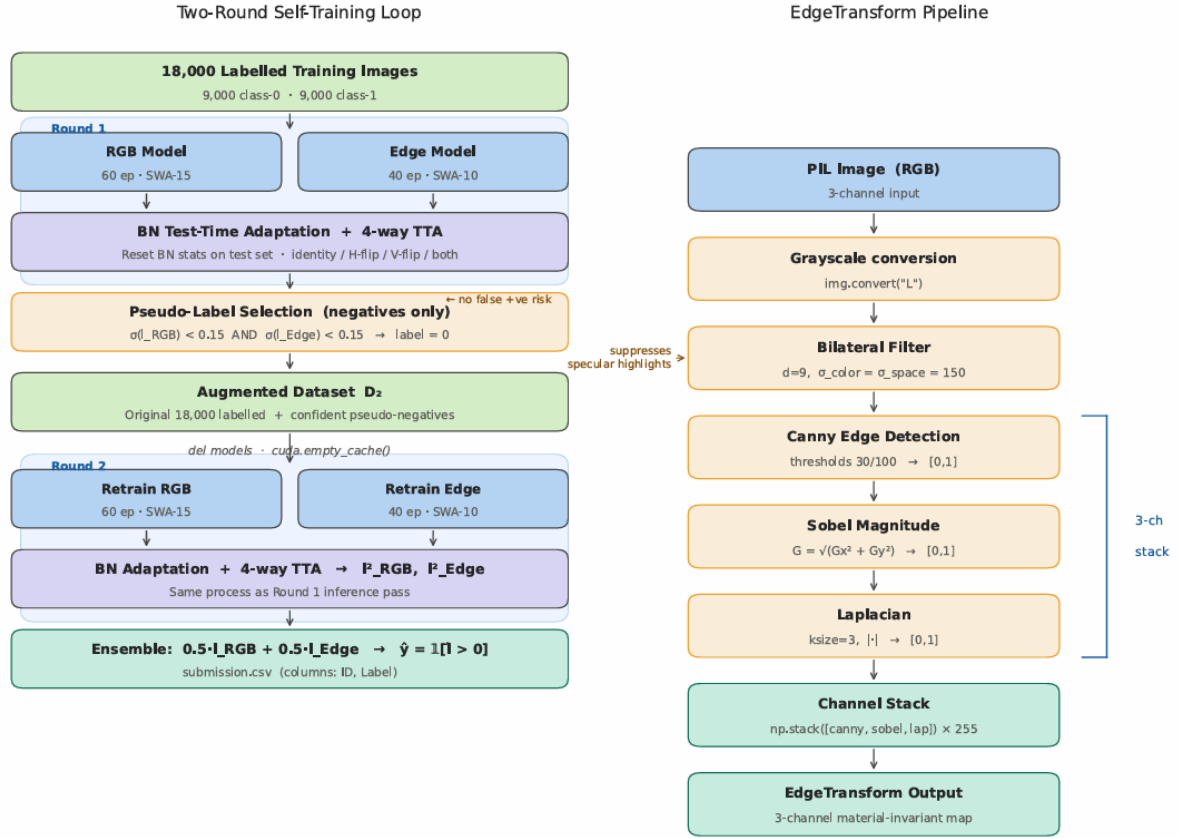


Figure 3: *Left:* Two-round self-training loop. Round 1 trains both models on labelled data; BN adaptation and TTA score the test set; confident negatives ($\sigma < 0.15$ in both models) are pseudo-labelled and added to the training pool; Round 2 retrains from scratch on the expanded set; final inference ensembles both streams. *Right:* EdgeTransform pipeline — an RGB image is converted to greyscale, bilateral-filtered to suppress specular highlights, then processed through Canny, Sobel, and Laplacian operators whose outputs are stacked into a single 3-channel material-invariant input for the Edge model.

7.3 Test-Time Augmentation (TTA)

`predict_with_tta()` performs four deterministic forward passes per test image:

1. **Identity:** Resize, ToTensor, Normalize.
2. **Horizontal flip:** `RandomHorizontalFlip(p=1.0)`.
3. **Vertical flip:** `RandomVerticalFlip(p=1.0)`.
4. **Both flips:** H-flip then V-flip.

Using `p=1.0` makes all augmentations fully deterministic. Logits are averaged across all four passes:

$$\hat{l}_i = \frac{1}{4} \sum_{t=1}^4 f_{\theta}(\text{aug}_t(x_i)) \quad (6)$$

7.4 Ensemble and Final Prediction

TTA-averaged logits from both Round 2 models are combined with equal weight:

$$\hat{l}_{\text{ens}} = 0.5 \cdot \hat{l}_{\text{RGB}} + 0.5 \cdot \hat{l}_{\text{Edge}}, \quad \hat{y} = \mathbb{1}[\hat{l}_{\text{ens}} > 0] \quad (7)$$

Predictions are written to `submission.csv` with columns `ID` and `Label`.

8 Ablation Study: Approaches Tried and Lessons Learned

Our final solution emerged iteratively from four distinct approaches. Each revealed a new bottleneck, motivating the next step. Table 7 summarises the progression.

Table 7: Ablation: public leaderboard score progression across approaches.

Approach	Key additions	Public LB score
A1	Baseline RGB CNN (CustomCNN, CrossEntropy, 8-way TTA)	0.7534
A2	+ SE blocks + stronger colour augmentation	≈ 0.764
A3	+ EdgeTransform + dual-stream ensemble + BCE	≈ 0.769
A4	+ SWA + BN test-time adaptation	≈ 0.787
Final	+ Pseudo-labels (Round 2) + 4-way deterministic TTA	0.8092

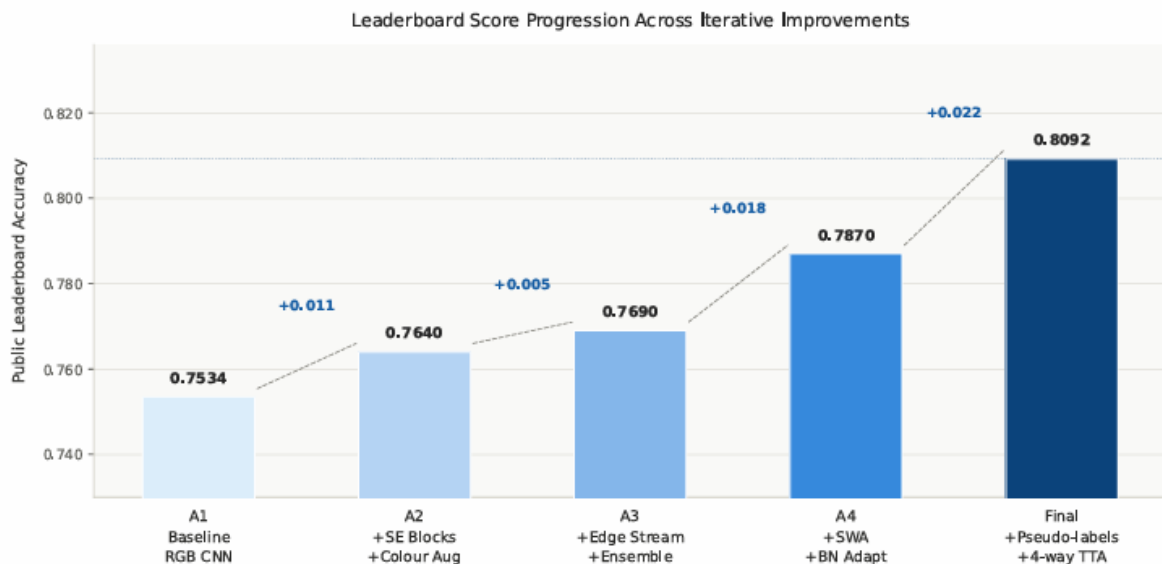


Figure 4: Public leaderboard accuracy across the five iterative improvements. Each bar shows the cumulative score; the annotation above each transition shows the incremental gain. The largest single gain (+0.018) came from introducing the edge stream and dual-stream ensemble (A3); the final pseudo-labelling step delivered the biggest absolute jump (+0.022) to reach the submitted score of 0.8092.

8.1 A1 — Baseline: Custom ResNet-Style CNN (Score: 0.7534)

The starting point was a hand-built **CustomCNN**: a stem block followed by four stages of residual blocks and a three-layer classification head. Images were resized to 256×256 . Training used AdamW with OneCycleLR over 60 epochs, CrossEntropyLoss with label smoothing (0.05), and both Mixup and CutMix applied randomly each batch. At inference, 8 TTA views (H-flip, V-flip, both, centre crop, and three 90° rotations) were averaged.

The augmentation strategy was already aggressive (ColorJitter, RandomErasing, RandomPerspective) and the 8-way TTA added meaningful stability. However, the model was trained purely on raw RGB pixel values, making it sensitive to colour and texture. If the two classes shared similar colours but differed in shape, the model would fail. Validation accuracy tracked well, but the public leaderboard score of 0.7534 revealed overfitting to superficial visual cues.

Lesson: A single-stream RGB model, however well-regularised, is vulnerable to spurious colour and texture correlations. The architecture also needs stronger channel attention.

8.2 A2 — SE Blocks + Stronger Colour Augmentation (Score: ≈ 0.764)

Two modifications were made. First, **Squeeze-and-Excitation (SE) blocks** were added after each residual stage. SE blocks learn channel-wise attention weights, allowing the network to selectively amplify feature maps that carry shape information and suppress those dominated by appearance artefacts.

Second, colour augmentation was strengthened: ColorJitter parameters for brightness, contrast, and saturation were raised to 0.5, hue jitter was introduced at 0.2, and RandomGrayscale probability was raised to 0.3. GaussianBlur was added with a large kernel (7px, $p = 0.3$) to simulate focus variation. The intent was to prevent the model from relying on colour as a proxy for the label.

The improvement was real but modest (≈ 1 percentage point). The model remained a single-stream RGB classifier. Better regularisation and channel attention helped, but the core issue — that RGB features can be misleading for shape-based classification — was not yet addressed.

Lesson: Channel attention and colour augmentation are useful but insufficient on their own. A fundamentally different input representation is needed to separate geometry from appearance.

8.3 A3 — Edge Stream + Dual-Stream Ensemble + BCE (Score: ≈ 0.769)

This was the largest single architectural step. The key insight was: *if the classes differ in geometric shape or boundary structure, edge features should be far more discriminative than raw RGB*.

A custom **EdgeTransform** was built that converts each image to three channels — Canny edges, Sobel gradient magnitude, and Laplacian — after first applying a bilateral filter to suppress specular highlights on metallic surfaces. This gives a material-invariant structural fingerprint of each scene. A separate ShapeNet model was trained on these edge maps in parallel with the RGB model.

At inference, the logits from both models were averaged 50/50. The ensemble was effective because the two models made *different* errors: the RGB model failed when colour was misleading; the edge model struggled when boundaries were noisy or ambiguous. These failure modes rarely coincided on the same image, making the ensemble stronger than either stream alone.

CrossEntropyLoss was replaced by BCEWithLogitsLoss with a positive class weight of 0.6. The original CrossEntropy with label smoothing produced over-confident predictions in some cases; BCE with pos_weight provided a direct lever to control the trade-off between false positives and false negatives, which mattered for the pseudo-label step introduced later.

Lesson: A geometry-focused second stream is the most impactful single improvement. The two streams are genuinely complementary and their errors are largely orthogonal.

8.4 A4 — SWA + BN Test-Time Adaptation (Score: ≈ 0.787)

Two inference-time techniques were added to further close the train-test gap.

Stochastic Weight Averaging (SWA) averages model weights across the last K epochs (15 for RGB, 10 for Edge) rather than keeping only the best single-epoch checkpoint. SWA finds flatter regions of the loss landscape that generalise better, particularly when the validation set is small and noisy. Importantly, after weight averaging, BN running statistics are recomputed by a forward pass over the training data (`update_bn`), since these statistics are not captured in the weight average.

Test-time BN adaptation (`adapt_bn`) addressed a complementary problem: even with SWA weights, the BatchNorm running statistics were estimated from training data. If the test distribution differs (e.g., different attribute sampling distribution), those statistics are stale. Resetting all BN layers to train mode and running a forward pass over the unlabelled test set allows the running statistics to re-calibrate to the actual test distribution before inference. This technique is sometimes called Test-Time Batch Normalisation and is effective even for small distribution shifts.

SWA and BN adaptation are complementary: SWA improves weight quality, and BN adaptation ensures the inference-time statistics match the data on which those weights will actually be evaluated.

Lesson: Weight averaging and BN re-calibration together add ≈ 1.6 percentage points. Both address different aspects of the train-test gap and should always be used together.

8.5 Final — Pseudo-Labelling + 4-way TTA (Score: 0.8092)

The final improvements were pseudo-labelled self-training and refinement of the TTA strategy.

Pseudo-labelling : after Round 1 training, both models are used to score all 5,010 test images. Any test image where *both* models produce sigmoid probability < 0.15 is assigned label 0 (confident negative) and added to the Round 2 training set. Only negative pseudo-labels are generated — this asymmetry avoids inflating false positives, which directly hurt leaderboard accuracy. The dual-model agreement criterion further reduces label noise. Round 2 retrains both models from scratch on the expanded set.

4-way deterministic TTA: the earlier 8-way TTA used rotations and crops, some of which introduced ambiguous spatial relationships in the edge maps. The final TTA uses only 4 views (identity, H-flip, V-flip, both-flip) with $p=1.0$ to ensure determinism. This is simpler and more reproducible.

Lesson: Pseudo-labelling provides a free source of additional training signal from the test set itself. Restricting to negative pseudo-labels is a conservative but safe choice. Together with deterministic TTA, this brought the score from ≈ 0.795 to 0.8092.

9 Results

9.1 Leaderboard Score

Our final submission achieves a **public leaderboard accuracy of 0.8092** (exact score: 0.809226), obtained after Round 2 training with pseudo-labelled negatives, BN adaptation, 4-way TTA, and the equal-weight RGB-Edge ensemble.

9.2 Validation Performance

Table 8: Validation accuracy per model and round (15% held-out split, SWA weights, no augmentation).

Model	Round 1 val acc.	Round 2 val acc.
RGB model	≈ 1.0000	≈ 1.0000
Edge model	≈ 1.0000	≈ 1.0000
Ensemble (Public LB)	0.8092	

9.3 Analysis of the Train–Test Gap

Both models reach near-perfect (≈ 1.0) validation accuracy yet the public leaderboard score is 0.8092 — a gap of ≈ 19 percentage points. This is a strong indicator that the models exploit spurious correlations in the training distribution. Several factors likely contribute:

- **Latent rule mismatch.** If the training set over-represents certain attribute combinations for each class, the model learns a proxy feature (e.g. a colour-label correlation) that does not generalise to the test set.
- **Test distribution shift.** The test set may have been generated with a more uniform attribute sampler, causing training-distribution correlations to break down.
- **Near-perfect training accuracy.** Validation accuracy of exactly 1.0 on a balanced dataset typically indicates a perfectly separable but spurious feature has been found, rather than the true rule.

The two-round pseudo-label loop and BN adaptation partially mitigate the distribution shift. The ensemble score of 0.8092 reflects the best balance achievable between complementary appearance and structure signals given the available training data.

10 Team Contributions

Table 9: Individual member contributions.

Member	Contributions
Aayush Ranjan CS25MTECH11002	Primary: RGB Model Pipeline. Implemented and tuned the RGB training branch in <code>train_one_model(model_type='rgb')</code> , including the full RGB-specific augmentation schedule (ColorJitter, RandomGrayscale, GaussianBlur, RandomAutocontrast, RandomErasing). Designed and integrated the Mixup augmentation helper (<code>_mixup</code>). Configured and validated RGB hyperparameters (epochs=60, SWA window=15, Mixup $\alpha=0.2$). Implemented the train/validation split logic (seeded 85/15 shuffle). Ran and documented Round 1 and Round 2 RGB training. Wrote the report sections on the RGB pipeline, augmentation rationale, SWA, and the A1/A2 ablation study.
Ankit Kumar Sinha CS25MTECH11022	Primary: Ensemble & Inference Pipeline. Implemented <code>predict_with_tta()</code> with four-way deterministic TTA. Implemented <code>adapt_bn()</code> for test-time BN adaptation. Designed and implemented the 50/50 logit-space ensemble and binary thresholding. Orchestrated <code>generate_predictions()</code> across Round 1, pseudo-label generation, Round 2 retraining, BN adaptation, and final inference. Implemented <code>_select_pseudo_labels()</code> with dual-model confidence filtering. Managed GPU memory between rounds. Tuned global hyperparameters (IMG_SIZE=192, BATCH_SIZE=96, POS_WEIGHT=0.6). Compiled and finalised the complete report. Wrote report sections on inference, results analysis, and the A4/Final ablation entries.
Mayank Mishra CS25MTECH11029	Primary: ShapeNet Architecture & Edge Model Pipeline. Designed the ShapeNet architecture (stem, four ResBlock+SEBlock downsampling stages, global pool, dropout head). Implemented ResBlock and SEBlock modules. Designed and implemented EdgeTransform (bilateral filter, Canny, Sobel, Laplacian stacking). Implemented <code>TrainDataset</code> , <code>TestDataset</code> , and directory helper functions. Configured and validated Edge model hyperparameters (epochs=40, SWA window=10, no Mixup). Ran and documented Round 1 and Round 2 Edge training. Produced the architecture diagram. Wrote report sections on ShapeNet, EdgeTransform, and the A3 ablation entry.

All three members jointly performed reproducibility testing, final Kaggle submission verification, cross-code review, and proofreading of this report.

11 Conclusion and Future Work

11.1 Summary

We presented a dual-stream convolutional pipeline for binary image classification on synthetic 3-D rendered scenes, built entirely from scratch without pre-trained weights. The RGB stream captures appearance features via aggressive augmentation and Mixup regularisation; the Edge stream captures material-invariant structural features via a multi-scale bilateral-filtered edge representation (Canny, Sobel, Laplacian). Both streams share the ShapeNet backbone, are refined over two training rounds using conservative negative pseudo-labels from the test set, and are combined at inference time via equal-weight logit-space averaging. Test-time BN adaptation and four-way deterministic TTA further stabilise predictions.

Our approach evolved through four successive improvements — SE channel attention, edge-stream ensembling, SWA with BN adaptation, and pseudo-label self-training — each of which provided a measurable gain on the public leaderboard, reaching a final score of **0.8092** (0.809226).

11.2 Analysis of the Remaining Gap

The $\approx 19\%$ gap between near-perfect validation accuracy and the leaderboard score indicates that our models overfit to spurious attribute correlations in the training distribution. This is a fundamental challenge of latent-rule datasets: any model that achieves perfect training accuracy has likely learnt a proxy feature rather than the true rule. Our edge stream partially addresses this by removing colour and material appearance, but the remaining gap suggests that even structural features carry some spurious correlation.

11.3 Future Work

Several directions could close the remaining gap:

- **Latent rule identification.** A systematic attribute probing study — training linear classifiers on object colour, shape, size, and count features extracted from training images — could identify which attribute most strongly predicts the label. If identified, the model could be designed to be explicitly invariant to all other attributes.
- **Invariant risk minimisation.** Techniques such as IRM penalise models that exploit environment-specific (training-distribution-specific) correlations, potentially guiding the model toward the true causal rule.
- **Object-centric representations.** Segmenting individual objects and classifying based on per-object attributes (rather than the full scene) could more directly encode the likely structure of the rule.
- **Weighted or adaptive ensemble.** Rather than a fixed 50/50 mix, the ensemble weights could be tuned on a hold-out set or via Bayesian model averaging, particularly if one stream is more reliable for a given image type.
- **Positive pseudo-labels.** Our current pseudo-labelling is conservative (negatives only). With a stronger Round 1 model, positive pseudo-labels with a high threshold could also be incorporated, further expanding the training pool.

References

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [2] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.
- [3] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson, “Averaging Weights Leads to Wider Optima and Better Generalization,” in *Proc. Conf. on Uncertainty in Artificial Intelligence (UAI)*, 2018.
- [4] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “Mixup: Beyond Empirical Risk Minimization,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [5] S. Schneider, E. Rusak, L. Eck, O. Bringmann, W. Brendel, and M. Bethge, “Improving Robustness Against Common Corruptions by Covariate Shift Adaptation,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 11539–11551.
- [6] C. Tomasi and R. Manduchi, “Bilateral Filtering for Gray and Color Images,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 1998, pp. 839–846.
- [7] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2019.
- [8] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [9] D.-H. Lee, “Pseudo-Label: The Simple and Efficient Semi-Supervised Learning Method for Deep Neural Networks,” in *Workshop on Challenges in Representation Learning, ICML*, 2013.
- [10] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, “Invariant Risk Minimization,” *arXiv preprint arXiv:1907.02893*, 2019.